

GoalWise

A goal-first budgeting app that tells you what's safe to spend, not just what you spent

1. Introduction

Most budgeting apps are built around categories: how much you spent on food, on entertainment, on transportation. They're excellent at telling you where your money went last month, but they leave the harder question unanswered — am I actually going to hit my savings goal? GoalWise flips the model. Instead of asking users to manage a dozen category limits, it asks for a single goal (for example, "save \$3,000 by December") and works backward to tell the user exactly how much they can safely spend each week to stay on pace. When spending drifts off track, GoalWise recalculates automatically and surfaces one clear action, rather than a dashboard of numbers the user has to interpret themselves.

2. Vision and Competitive Analysis

Our vision is a budgeting tool that feels forward-looking instead of retrospective. Traditional tools (Mint, YNAB) report on what already happened and ask users to manually rebuild their budget every time income or spending changes — a tedious process that is especially painful for users with irregular income, like students juggling internships and part-time work. GoalWise is built around a single outcome rather than a set of categories, and it stays current automatically: every new transaction updates the weekly "safe to spend" number and recalculates the pace needed to still hit the goal on time.

This distinguishes GoalWise from existing tools in a concrete way. Category-based apps answer "how did I do last month?" GoalWise answers "am I still on pace, and what do I need to do this week?" — a question that's more motivating and more actionable for someone trying to hit a specific financial target.

3. Proposed Technical Approach

GoalWise will be built as a standard three-tier web application, kept deliberately simple to fit a six-week timeline:

- Frontend: React (or Next.js) — goal setup form, weekly pace dashboard, and progress visualization.
- Backend: Python (FastAPI) — hosts the goal-math engine and transaction-processing endpoints.
- Database: PostgreSQL or SQLite — stores user goals, transactions, and weekly recalculation history.

- Data input: CSV upload or bundled sample transaction data for the MVP. We are deliberately avoiding live bank sync (e.g., Plaid) — integrating real banking credentials introduces compliance and security overhead that isn't realistic to handle safely within a class project.
- Core logic: A deterministic goal-math engine (safe-to-spend calculation, weekly pace recalculation) that is unit-tested independently of any AI component, so the numbers users rely on are reproducible and verifiable.

Use of AI in Development

AI will be used to support the product experience, not to perform the core financial calculations. Specifically, we plan to use an LLM layer to generate plain-English weekly check-ins and nudges (e.g., "you're \$40 ahead of pace this week"), to classify raw transaction descriptions into spending categories, and to suggest where a user might trim spending when they fall off pace, along with a short explanation of the reasoning. The safe-to-spend and pace-recalculation math itself will remain deterministic and independently tested, so the numbers the app is built around don't depend on AI output. We also expect to use AI coding assistants (e.g., Claude, OpenAI, Copilot etc..) during development to help scaffold components, debug, and write test cases, with all generated code reviewed and understood by the team before it ships.

4. Most Serious Risk and Mitigation

Risk: an overly optimistic safe-to-spend calculation. The entire premise of GoalWise depends on trusting the number it gives you each week. If that number is calculated too generously — for example, not accounting properly for upcoming recurring expenses or irregular income — a user could follow it in good faith and still end up overspending and missing their goal. This isn't a generic engineering risk; it's specific to a product whose core value proposition is a single trusted number, so getting that number wrong undermines the product itself, not just one feature.

Mitigation:

- Build a conservative buffer into the calculation rather than an exact break-even estimate.
- Recalculate pace weekly (not just at month-end), so users get early corrections instead of a surprise shortfall close to the goal date.
- Display the assumptions and formula behind every calculated limit, so users can see why a number changed, not just that it changed.
- Limit the MVP to a small number of clearly defined goal and expense structures rather than trying to generalize to every financial situation on day one.

- Write automated tests around the goal-math engine specifically, using edge cases like irregular income and missed payments.

5. The team consists of:

Vishal — Product Lead | Nati — Technical Lead | Ashutosh — UI/UX & Market Research | Thanh — QA & Documentation